

Corso Python

Modulo 5: Python nel Mondo Reale & Data Analysis

Collegamento al Modulo Precedente

Nel **Modulo 4** abbiamo imparato:

-  Gestione eccezioni: `try`, `except`, `finally`
-  Moduli e import
-  OOP: classi, ereditarietà, polimorfismo

Ora usciamo dalla sandbox:

-  **venv & pip:** Gestire dipendenze professionalmente
-  **File I/O:** Leggere/scrivere dati persistenti
-  **API & Pandas:** Interagire col mondo esterno

Agenda del Modulo (3 Ore)

Usciamo dalla "sandbox" per interagire con il sistema e i dati esterni.

1. **Virtual Environments:** Isolare i progetti e le dipendenze.
2. **PIP & PyPI:** Scaricare librerie dal mondo (Pandas, Requests).
3. **File I/O:** Leggere e scrivere file TXT, CSV e JSON.
4. **Intro a Pandas:** Primi passi nell'analisi dati.
5. **Laboratorio:** Analisi log di sistema.

1. Virtual Environments & PIP

Gestione professionale dei progetti

Cos'è un Virtual Environment?

Un **virtual environment** (ambiente virtuale) è una **copia isolata** dell'interprete Python con le proprie librerie installate, completamente indipendente dal sistema.

Perché è fondamentale?

Nello sviluppo software professionale, ogni progetto ha esigenze diverse: versioni specifiche di librerie, dipendenze particolari, configurazioni uniche. Senza isolamento, questi requisiti entrano in conflitto.

Analogia: Pensa a ogni progetto come a un appartamento separato. Ogni appartamento ha i propri mobili (librerie) senza interferire con gli altri.

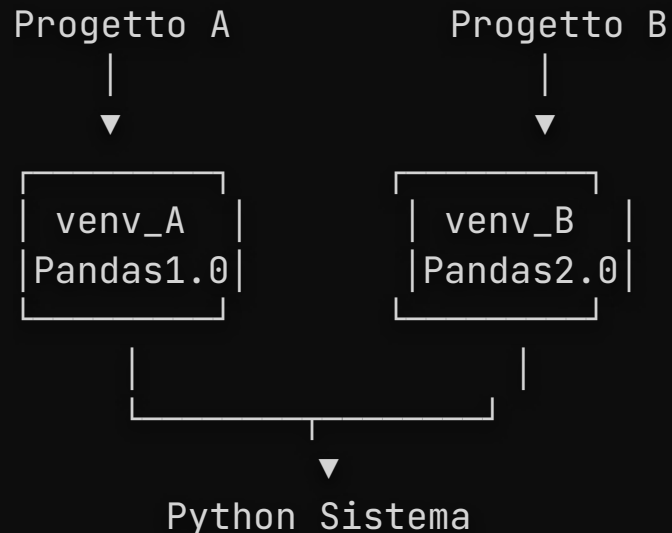
Vantaggi dei virtual Environments

Senza venv	Con venv
Tutte le librerie in un unico posto	Ogni progetto ha le sue librerie
Conflitti di versione inevitabili	Isolamento completo
Difficile replicare l'ambiente	Facile condividere con il team
"Funziona sul mio PC"	Funziona ovunque

Perché servono i Virtual Environments?

Immagina di avere due progetti, che richiedono due versioni di pandas diverse:

Se installi tutto globalmente nel PC, avrai conflitti (**Dependency Hell**).



Creare un Virtual Environment

Comandi da terminale (non da console Python!):

```
# Naviga nella cartella del progetto
cd mio_progetto

# Crea il virtual environment
python -m venv venv

# Cosa viene creato:
mio_progetto/
├── venv/
│   ├── Scripts/      # (Windows) o bin/ (Linux/Mac)
│   ├── Lib/          # Librerie installate
│   ├── Include/
│   └── pyenv.config  # Configurazione
└── main.py
```

Convenzione: Chiama la cartella `venv` o `.venv` (con punto per nasconderla).

Attivare e Disattivare il Venv

```
# ATTIVAZIONE
# Windows (CMD):
venv\Scripts\activate

# Windows (PowerShell):
.\venv\Scripts\Activate.ps1

# Mac/Linux:
source venv/bin/activate
```

Quando attivo, vedrai `(venv)` all'inizio della riga di comando e potrai iniziare a installare i pacchetti:

```
(venv) C:\mio_progetto>
```

```
# DISATTIVAZIONE (uguale ovunque)
deactivate
```

PIP: Il Package Manager di Python

PIP (Pip Installs Packages) è lo strumento ufficiale per gestire le librerie Python. È il "corriere" che scarica e installa software dal repository centrale.

PyPI (Python Package Index) è il repository ufficiale con **500.000+** pacchetti open source, mantenuto dalla Python Software Foundation.

Come funziona il processo:

1. Scrivi `pip install nome_pacchetto`
2. PIP cerca il pacchetto su PyPI
3. Scarica il pacchetto e tutte le sue dipendenze
4. Installa tutto nel venv attivo (o globalmente se nessun venv è attivo)

```
# Verifica versione pip  
pip --version
```

Comandi PIP Essenziali

Aggiorna pip (consigliato)

```
python -m pip install --upgrade pip
```

Installare una libreria

```
pip install pandas
```

Installare versione specifica

```
pip install pandas==2.0.0
```

Installare versione minima

```
pip install "pandas ≥ 1.5.0"
```

Installare più librerie

```
pip install pandas numpy matplotlib
```

Vedere cosa è installato

```
pip list
```

Info dettagliate su un pacchetto

```
pip show pandas
```

Disinstallare

```
pip uninstall pandas
```

Requirements.txt: Condividere l'Ambiente

Il file `requirements.txt` elenca tutte le dipendenze del progetto.

```
# CREARE il file (esporta l'ambiente attuale)  
pip freeze > requirements.txt
```

```
# Contenuto di requirements.txt  
pandas==2.0.0  
numpy==1.24.0
```

```
# RIPRISTINARE l'ambiente (su un altro PC o per un collega)  
pip install -r requirements.txt
```

Best Practice: Includi `requirements.txt` nel repository Git, **escludi** la cartella `venv/`.

.gitignore per Progetti Python

Crea un file `.gitignore` nella root del progetto:

```
# Virtual Environment
venv/

# Cache Python
__pycache__/
*.pyc
*.pyo

# IDE
.vscode/

# File di sistema
.DS_Store
```

Perché escludere venv? È ricostruibile da `requirements.txt` e può pesare centinaia di MB.



Esercizi Intermedi: Virtual Environments & PIP

Pratica Prima di Proseguire

Esercizio 1.1: Creare e Attivare un Venv

Obiettivo: Crea un virtual environment per un nuovo progetto e installa una libreria.

Passi:

1. Crea una cartella `progetto_test`
2. Naviga nella cartella
3. Crea un virtual environment chiamato `venv`
4. Attiva il virtual environment
5. Installa la libreria `requests`
6. Verifica che sia installata con `pip list`

Esercizio 1.2: Gestire Requirements

Obiettivo: Esporta e ripristina le dipendenze di un progetto.

```
# Scenario: Hai installato pandas, numpy e matplotlib nel tuo venv
# 1. Esporta le dipendenze in requirements.txt
# 2. Disattiva il venv
# 3. Elimina la cartella venv
# 4. Ricrea il venv
# 5. Riattiva il venv
# 6. Ripristina le dipendenze da requirements.txt
```

Comandi da usare:

```
# Esporta
pip freeze > requirements.txt

# Ripristina
pip install -r requirements.txt
```


Esercizio 1.3: Installare Versioni Specifiche

Obiettivo: Installa versioni specifiche di librerie per evitare conflitti.

Task:

1. Installa `pandas` versione `1.5.0` (non l'ultima)
2. Installa `numpy` con versione minima `1.20.0`
3. Verifica le versioni installate con `pip show pandas` e `pip show numpy`
4. Crea un `requirements.txt` con le versioni esatte

Comandi:

```
pip install pandas=1.5.0
pip install "numpy ≥ 1.20.0"
pip show pandas
```

2. File I/O

Leggere e Scrivere Dati

Perché il File I/O è Importante?

I programmi reali devono lavorare su dati **persistenti**:

Senza File I/O	Con File I/O
Dati persi alla chiusura	Dati salvati permanentemente
Nessuna comunicazione	Scambio dati tra programmi
Input solo da tastiera	Input da file di configurazione
Output solo a schermo	Report, log, export

Formati comuni:

- **TXT:** Testo semplice, log, note
- **CSV:** Dati tabulari (Excel-like)
- **JSON:** Dati strutturati, API web
- **XML:** Configurazioni, dati legacy

La Funzione open()

`open(file, mode, encoding)` è la funzione base per lavorare con i file.

Modalità	Significato	Se il file non esiste
<code>'r'</code>	Read (lettura)	Errore
<code>'w'</code>	Write (scrittura)	Lo crea
<code>'a'</code>	Append (aggiunta)	Lo crea
<code>'x'</code>	Create (creazione esclusiva)	Errore se esiste
<code>'r+'</code>	Read + Write	Errore
<code>'b'</code>	Binary mode (es. <code>'rb'</code>)	-

Sempre specificare `encoding="utf-8"` per evitare problemi con caratteri speciali (àèéù).

Context Manager: with open()

Il **Context Manager** è un pattern Python che garantisce la corretta gestione delle risorse. La keyword `with` definisce un blocco dove la risorsa è disponibile.

Perché è importante?

- I file aperti consumano risorse del sistema operativo
- Se il programma crasha, i file non chiusi possono corrompersi
- Il context manager chiude **sempre** il file, anche se c'è un'eccezione

```
# MODO CORRETTO: Context Manager
with open("note.txt", "w", encoding="utf-8") as f:
    f.write("Prima riga del file.\n")
    f.write("Seconda riga.")

# MODO SBAGLIATO (da evitare)
f = open("note.txt", "w")
f.write("Testo")
f.close() # Rischio: se c'è errore prima, file rimane aperto!
```

Lettura File di Testo

```
# Leggere TUTTO il contenuto
with open("note.txt", "r", encoding="utf-8") as f:
    contenuto = f.read()
    print(contenuto)

# Leggere riga per riga (lista)
with open("note.txt", "r", encoding="utf-8") as f:
    righe = f.readlines() # Lista di stringhe
    for riga in righe:
        print(riga.strip()) # strip() rimuove \n

# Iterare direttamente (più efficiente per file grandi)
with open("log.txt", "r", encoding="utf-8") as f:
    for riga in f: # Non carica tutto in memoria
        print(riga.strip())
```

Scrittura File di Testo

```
# Scrittura (sovrascrive tutto)
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Riga 1\n")
    f.write("Riga 2\n")

# Append (aggiunge alla fine)
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("Nuova entry nel log\n")

# Scrivere più righe da lista
righe = ["Primo", "Secondo", "Terzo"]
with open("lista.txt", "w", encoding="utf-8") as f:
    f.writelines(riga + "\n" for riga in righe)
```

Gestire i Percorsi con pathlib

Il modulo `pathlib` rende la gestione dei percorsi cross-platform.

```
from pathlib import Path

# Creare percorsi (funziona su Windows e Linux)
cartella = Path("dati")
file_path = cartella / "vendite.csv" # dati/vendite.csv

# Verifiche
print(file_path.exists())           # True/False
print(file_path.is_file())          # È un file?
print(file_path.is_dir())           # È una cartella?

# Creare cartelle
cartella.mkdir(exist_ok=True) # Crea se non esiste

# Leggere/scrivere direttamente
file_path.write_text("Contenuto", encoding="utf-8")
contenuto = file_path.read_text(encoding="utf-8")
```


File CSV: Il Modulo csv

CSV (Comma Separated Values) è il formato più comune per dati tabulari.

```
import csv

# SCRITTURA CSV
with open("prodotti.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Nome", "Prezzo", "Quantità"]) # Header
    writer.writerow(["Laptop", 999.99, 10])
    writer.writerow(["Mouse", 29.99, 50])

# LETTURA CSV
with open("prodotti.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader) # Salta l'header
    for riga in reader:
        nome, prezzo, qta = riga
        print(f"{nome}: €{prezzo}")
```

CSV con Dizionari (DictReader/DictWriter)

Più leggibile quando hai molte colonne.

```
import csv

# SCRITTURA con dizionari
prodotti = [
    {"nome": "Laptop", "prezzo": 999.99, "qta": 10},
    {"nome": "Mouse", "prezzo": 29.99, "qta": 50}
]

with open("prodotti.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["nome", "prezzo", "qta"])
    writer.writeheader()
    writer.writerows(prodotti)

# LETTURA con dizionari
with open("prodotti.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for riga in reader:
        print(f"{riga['nome']}: €{riga['prezzo']}")
```

Gestire i JSON

JSON (JavaScript Object Notation) è diventato il formato **universale** per lo scambio dati sul web. Nonostante il nome, è indipendente dal linguaggio.

Perché JSON è così popolare?

- **Leggibile:** Testo semplice, comprensibile agli umani
- **Leggero:** Meno verbose di XML
- **Universale:** Supportato da tutti i linguaggi
- **Standard:** Usato da quasi tutte le API REST

Corrispondenza tipi Python ↔ JSON

Python	JSON
<code>dict</code>	<code>object {}</code>
<code>list</code>	<code>array []</code>
<code>str</code>	<code>string</code>
<code>int/float</code>	<code>number</code>
<code>True/False</code>	<code>true/false</code>
<code>None</code>	<code>null</code>

JSON: Lettura e Scrittura

```
import json

dati = {
    "nome": "Mario",
    "skills": ["Python", "SQL"],
    "attivo": True,
    "eta": None
}

# SCRIVERE su file (dump)
with open("dati.json", "w", encoding="utf-8") as f:
    json.dump(dati, f, indent=4, ensure_ascii=False)

# LEGGERE da file (load)
with open("dati.json", "r", encoding="utf-8") as f:
    dati_caricati = json.load(f)
    print(dati_caricati["skills"][0]) # → "Python"
```

`indent=4` rende il JSON leggibile, `ensure_ascii=False` preserva accenti.

JSON: Stringhe (dumps/loads)

Per convertire tra dizionari Python e stringhe JSON:

```
import json

dizionario = {"nome": "Luigi", "punti": 100}
# Dizionario → Stringa JSON
json_string = json.dumps(dizionario, indent=2)
print(json_string)
# {
#   "nome": "Luigi",
#   "punti": 100
# }

# Stringa JSON → Dizionario
json_ricevuto = '{"nome": "Peach", "punti": 200}'
dati = json.loads(json_ricevuto)
print(dati["nome"]) # → "Peach"
```

Nota: `dump` / `load` = file | `dumps` / `loads` = string



Esercizi Intermedi: File I/O

Pratica Prima di Proseguire

Esercizio 2.1: Gestione Log File

Obiettivo: Crea un sistema di logging semplice che scrive su file.

```
from datetime import datetime
from pathlib import Path

def scrivi_log(messaggio, livello="INFO"):
    """
    Scrive un messaggio di log nel file log.txt.
    Formato: [TIMESTAMP] LIVELLO: messaggio
    """
    # Implementa la funzione usando 'with open()' in modalità append
    # Usa datetime.now() per il timestamp
    pass

# Test:
# scrivi_log("Applicazione avviata")
# scrivi_log("Errore di connessione", livello="ERROR")
# scrivi_log("Operazione completata", livello="SUCCESS")
```


Esercizio 2.2: CSV di Prodotti

Obiettivo: Crea e leggi un file CSV con dati di prodotti.

```
import csv

# Parte 1: Scrivi i dati
prodotti = [
    {"nome": "Laptop", "prezzo": 999.99, "quantita": 5},
    {"nome": "Mouse", "prezzo": 29.99, "quantita": 20},
    {"nome": "Tastiera", "prezzo": 79.99, "quantita": 15}
]

# Scrivi in prodotti.csv usando DictWriter
# Usa newline="" per evitare righe vuote

# Parte 2: Leggi e calcola
# Leggi il CSV e calcola il valore totale dell'inventario
# (prezzo * quantita per ogni prodotto, poi somma)
```

Esercizio 2.3: Configurazione JSON

Obiettivo: Gestisci un file di configurazione in JSON.

```
import json
from pathlib import Path

# Crea un dizionario di configurazione
config = {
    "app_name": "MyApp",
    "version": "1.0.0",
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "mydb"
    },
    "debug": True
}

# 1. Salva in config.json (con indentazione)
# 2. Leggi il file
# 3. Modifica il valore di "debug" a False
# 4. Salva di nuovo
# 5. Stampa la configurazione aggiornata
```

3. Requests: Interagire con le API

Scaricare dati dal web

Cos'è un'API?

Un'**API** (Application Programming Interface) è un'interfaccia che permette a due software di comunicare tra loro.

API REST (le più comuni):

- Usano il protocollo HTTP (quello dei siti web)
- Scambiano dati in formato JSON
- Operazioni standard: GET (leggi), POST (crea), PUT (aggiorna), DELETE (elimina)

Esempi di API pubbliche:

- Meteo, mappe, social media
- Dati finanziari, notizie
- Servizi di traduzione, AI

Il Modulo requests

`requests` è la libreria standard de facto per le chiamate HTTP in Python. Rende semplice ciò che altrimenti richiederebbe decine di righe di codice.

```
pip install requests
```

```
import requests

# GET: Scaricare dati
response = requests.get("https://api.example.com/data")

# Verificare lo stato
print(response.status_code) # 200 = OK, 404 = Non trovato, 500 = Errore server
print(response.ok)         # True se 2xx (successo)

# Ottenere il contenuto
print(response.text)        # Come stringa grezza
print(response.json())      # Come dizionario Python (se è JSON)
```

Esempio: API Pubblica

```
import requests

# API gratuita per dati fake
url = "https://jsonplaceholder.typicode.com/users"

response = requests.get(url)

if response.ok:
    utenti = response.json()  # Lista di dizionari

    for utente in utenti[:3]:  # Primi 3
        print(f"{utente['name']} - {utente['email']}")
else:
    print(f"Errore: {response.status_code}")
```

Requests con Parametri

```
import requests

# Query parameters (GET)
params = {
    "q": "python",
    "limit": 10
}
response = requests.get("https://api.example.com/search", params=params)
# URL finale: https://api.example.com/search?q=python&limit=10

# Headers personalizzati
headers = {
    "Authorization": "Bearer TOKEN_123",
    "Content-Type": "application/json"
}
response = requests.get(url, headers=headers)

# POST con dati JSON
dati = {"nome": "Mario", "email": "mario@email.com"}
response = requests.post(url, json=dati)
```



Esercizi Intermedi: Requests & API

Pratica Prima di Proseguire

Esercizio 3.1: Scaricare Dati da API

Obiettivo: Scarica e processa dati da un'API pubblica.

```
import requests

# API: https://jsonplaceholder.typicode.com/posts
# Scarica tutti i post e:
# 1. Conta quanti post ci sono
# 2. Trova il post con il titolo più lungo
# 3. Stampa i primi 3 post (id, titolo)

# Suggerimento: response.json() restituisce una lista di dizionari
```

Esercizio 3.2: Gestire Errori HTTP

Obiettivo: Gestisci correttamente gli errori nelle chiamate API.

```
import requests

def scarica_utente(user_id):
    """
    Scarica i dati di un utente da JSONPlaceholder.
    URL: https://jsonplaceholder.typicode.com/users/{user_id}

    Gestisci:
    - Status code 200: restituisci i dati
    - Status code 404: restituisci "Utente non trovato"
    - Altri errori: restituisci "Errore di connessione"
    """
    # Implementa la funzione
    # Usa response.status_code per verificare lo stato
    pass

# Test:
# print(scarica_utente(1))    # Dovrebbe funzionare
# print(scarica_utente(999))  # Dovrebbe dare 404
```

Esercizio 3.3: Parametri di Query

Obiettivo: Usa parametri per filtrare i risultati di un'API.

```
import requests

# API: https://jsonplaceholder.typicode.com/comments
# Questa API accetta parametri di query come 'postId'

# Task:
# 1. Scarica tutti i commenti del post con id=1
#    (usa params={"postId": 1})
# 2. Conta quanti commenti ha ricevuto
# 3. Stampa l'email di chi ha scritto il primo commento

# Suggerimento:
# response = requests.get(url, params={"postId": 1})
```

4. Analizzare dati con Pandas

"Excel on Steroids"

Cos'è Pandas?

Pandas è la libreria fondamentale per l'analisi dati in Python. Creata nel 2008, è oggi usata da data scientist, analisti e sviluppatori in tutto il mondo.

Perché Pandas invece di Excel?

- **Scalabilità:** Gestisce milioni di righe (Excel si blocca a ~1 milione)
- **Automazione:** Script riproducibili vs operazioni manuali
- **Integrazione:** Si collega a database, API, altri strumenti Python
- **Potenza:** Operazioni complesse in una riga di codice

Strutture dati principali

Struttura	Descrizione	Analogia
Series	Array 1D con indice	Colonna Excel
DataFrame	Tabella 2D	Foglio Excel

```
pip install pandas
```

```
import pandas as pd  # Convenzione universale: alias 'pd'
```

DataFrame: La Struttura Chiave

Il **DataFrame** è il cuore di Pandas: una **tabella bidimensionale** in memoria, simile a un foglio Excel o una tabella SQL.

Caratteristiche:

- **Righe:** Identificate da un indice (numerico di default, ma personalizzabile)
- **Colonne:** Hanno nomi e tipi di dati (int, float, string, datetime...)
- **Eterogeneo:** Ogni colonna può avere un tipo diverso
- **Mutabile:** Puoi aggiungere/rimuovere righe e colonne

Esempio di DataFrame

```
import pandas as pd

# Creare DataFrame da dizionario (modo più comune)
dati = {
    "Nome": ["Mario", "Luigi", "Peach"],
    "Età": [35, 33, 32],
    "Città": ["Roma", "Milano", "Napoli"]
}

df = pd.DataFrame(dati)
print(df)
```

#	Nome	Età	Città
# 0	Mario	35	Roma
# 1	Luigi	33	Milano
# 2	Peach	32	Napoli

Leggere File con Pandas

```
import pandas as pd

# CSV (Comma Separated Values)
df = pd.read_csv("dati.csv")

# Excel
df = pd.read_excel("dati.xlsx", sheet_name="Foglio1")

# JSON
df = pd.read_json("dati.json")

# Da URL direttamente!
url = "https://example.com/dati.csv"
df = pd.read_csv(url)

# Parametri utili per CSV
df = pd.read_csv("dati.csv",
                  sep=";",          # Separatore
                  encoding="utf-8", # Encoding
                  index_col=0)      # Colonna indice
```

Ispezione del DataFrame

Prima di analizzare, **esplora** i dati:

```
# Prime/ultime righe
print(df.head())      # Prime 5 righe
print(df.head(10))    # Prime 10 righe
print(df.tail())      # Ultime 5 righe

# Dimensioni
print(df.shape)       # (righe, colonne)

# Informazioni struttura
print(df.info())      # Tipi, valori nulli, memoria

# Statistiche descrittive
print(df.describe())  # Media, std, min, max, quartili

# Nomi colonne
print(df.columns)     # Index(['Nome', 'Età', ...])
```

Selezione Colonne

```
# Singola colonna → Series  
nomi = df["Nome"]  
print(type(nomi))  # <class 'pandas.core.series.Series'>  
  
# Singola colonna (sintassi alternativa)  
nomi = df.Nome  # Funziona se il nome non ha spazi  
  
# Più colonne → DataFrame  
subset = df[["Nome", "Città"]]  
print(type(subset))  # <class 'pandas.core.frame.DataFrame'>
```

Selezione Righe: loc e iloc

```
#   Nome  Età   Città
# 0  Mario   35    Roma
# 1  Luigi   33   Milano
# 2  Peach   32   Napoli
```

```
# iloc: selezione per POSIZIONE (integer location)
print(df.iloc[0])           # Prima riga
print(df.iloc[0:2])         # Prime 2 righe
print(df.iloc[0, 1])        # Riga 0, colonna 1 → 35
```

```
# loc: selezione per ETICHETTA (label)
print(df.loc[0])            # Riga con indice 0
print(df.loc[0, "Nome"])    # → "Mario"
print(df.loc[0:1, ["Nome", "Città"]]) # Subset
```

Filtraggio con Condizioni

Molto più potente di Excel!

```
# Condizione singola
adulti = df[df["Età"] ≥ 18]

# Condizioni multiple (AND)
filtro = df[(df["Età"] > 30) & (df["Città"] = "Roma")]

# Condizioni multiple (OR)
filtro = df[(df["Città"] = "Roma") | (df["Città"] = "Milano")]

# Valori in una lista
città_interesse = ["Roma", "Milano"]
filtro = df[df["Città"].isin(città_interesse)]

# Filtrare stringhe
filtro = df[df["Nome"].str.contains("ar")] # Contiene "ar"
filtro = df[df["Nome"].str.startswith("M")] # Inizia con M
```

Creare e Modificare Colonne

```
# Nuova colonna calcolata
df["Età_tra_10_anni"] = df["Età"] + 10

# Colonna condizionale
df["Maggiorenne"] = df["Età"] ≥ 18

# Colonna con apply (funzione personalizzata)
def categoria_età(età):
    if età < 30:
        return "Giovane"
    elif età < 50:
        return "Adulto"
    else:
        return "Senior"

df["Categoria"] = df["Età"].apply(categoria_età)

# Rinominare colonne
df = df.rename(columns={"Età": "Age", "Nome": "Name"})
```

Gestire Valori Mancanti (NaN)

I dati reali sono **sempre** imperfetti. Celle vuote, errori di input, dati non disponibili: Pandas li rappresenta come **NaN** (Not a Number).

Perché gestire i NaN è critico?

- Molte operazioni matematiche falliscono con NaN
- I modelli di Machine Learning non accettano NaN
- Possono falsare statistiche e analisi

Strategie comuni per gestire i dati mancanti

- **Rimuovere:** Se pochi dati mancanti
- **Sostituire:** Con media, mediana, o valore fisso
- **Interpolare:** Per serie temporali

```
# Verificare valori nulli
print(df.isnull().sum()) # Conta NaN per colonna

# Rimuovere righe con NaN
df_clean = df.dropna() # Rimuove TUTTE le righe con NaN
df_clean = df.dropna(subset=["Età"]) # Solo se NaN in "Età"

# Sostituire NaN con un valore
df["Età"] = df["Età"].fillna(0) # Con zero
df["Età"] = df["Età"].fillna(df["Età"].mean()) # Con la media
```


Ordinamento

Ordinare per una colonna

```
df_ordinato = df.sort_values("Età")
```

Ordine decrescente

```
df_ordinato = df.sort_values("Età", ascending=False)
```

Ordinare per più colonne

```
df_ordinato = df.sort_values(["Città", "Età"],  
                             ascending=[True, False])
```

Ordinare per indice

```
df_ordinato = df.sort_index()
```

Aggregazioni e Groupby

Il **groupby** è una delle operazioni più potenti di Pandas. Permette di raggruppare dati per categoria e calcolare statistiche, come le **Pivot Table** di Excel ma molto più flessibili.

Paradigma Split-Apply-Combine:

1. **Split:** Divide i dati in gruppi basati su una colonna
2. **Apply:** Applica una funzione a ogni gruppo (sum, mean, count...)
3. **Combine:** Combina i risultati in un nuovo DataFrame

Aggregazione e Groupby

```
# Statistiche globali
print(df["Età"].mean())    # Media
print(df["Età"].sum())     # Somma
print(df["Età"].max())     # Massimo

# Groupby: raggruppare per categoria
per_città = df.groupby("Città")["Età"].mean()
print(per_città)
# Città
# Milano      33.0
# Napoli      32.0
# Roma        35.0

# Più aggregazioni contemporaneamente
stats = df.groupby("Città").agg({
    "Età": ["mean", "min", "max"],
    "Nome": "count"
})
```

Salvare i Dati

```
# CSV
df.to_csv("output.csv", index=False) # index=False evita colonna extra

# Excel
df.to_excel("output.xlsx", sheet_name="Dati", index=False)

# JSON
df.to_json("output.json", orient="records", indent=4)

# Più fogli Excel
with pd.ExcelWriter("report.xlsx") as writer:
    df1.to_excel(writer, sheet_name="Vendite")
    df2.to_excel(writer, sheet_name="Clienti")
```



Esercizi Intermedi: Pandas

Pratica Prima di Proseguire

Esercizio 4.1: Creare e Filtrare DataFrame

Obiettivo: Crea un DataFrame e applica filtri.

```
import pandas as pd

# Crea un DataFrame con questi dati
dati = {
    "Nome": ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "Età": [25, 30, 35, 28, 42],
    "Città": ["Roma", "Milano", "Roma", "Napoli", "Milano"],
    "Stipendio": [30000, 45000, 50000, 35000, 60000]
}

df = pd.DataFrame(dati)

# Task:
# 1. Mostra solo le persone con età > 30
# 2. Mostra solo le persone di Milano
# 3. Mostra nome e stipendio delle persone di Roma
# 4. Calcola lo stipendio medio per città (usa groupby)
```

Esercizio 4.2: Analisi CSV

Obiettivo: Carica e analizza un CSV (crealo prima).

```
import pandas as pd

# Parte 1: Crea il CSV
vendite = {
    "Prodotto": ["Laptop", "Mouse", "Tastiera", "Monitor", "Laptop"],
    "Quantità": [2, 5, 3, 1, 1],
    "Prezzo": [999, 29, 79, 299, 999]
}

df = pd.DataFrame(vendite)
df.to_csv("vendite.csv", index=False)

# Parte 2: Analizza
# 1. Ricarica il CSV
# 2. Aggiungi colonna "Totale" (Quantità * Prezzo)
# 3. Trova il prodotto con il totale più alto
# 4. Calcola il fatturato totale (somma di tutti i Totale)
# 5. Raggruppa per Prodotto e somma le quantità vendute
```

Esercizio 4.3: Gestire Dati Mancanti

Obiettivo: Lavora con valori NaN.

```
import pandas as pd
import numpy as np

# DataFrame con valori mancanti
dati = {
    "Nome": ["Mario", "Luigi", "Peach", "Toad"],
    "Età": [35, np.nan, 32, 28],
    "Email": ["mario@email.it", "luigi@email.it", np.nan, "toad@email.it"],
    "Punteggio": [100, 85, np.nan, 90]
}

df = pd.DataFrame(dati)

# Task:
# 1. Identifica quanti valori NaN ci sono per colonna
# 2. Sostituisci i NaN in "Età" con la media delle età
# 3. Sostituisci i NaN in "Email" con "non_disponibile@email.it"
# 4. Rimuovi le righe con NaN in "Punteggio"
# 5. Salva il DataFrame pulito in "dati_puliti.csv"
```


Laboratorio 5

Data Processing Reale

Esercizio 1: Analisi Vendite

Scenario: Hai un file `vendite_raw.csv` con colonne: *Prodotto*, *Quantità*, *PrezzoUnitario*.
Alcuni prezzi sono negativi (errore di sistema).

Obiettivo:

1. Caricare il CSV con Pandas.
2. Filtrare via i prezzi errati (≤ 0).
3. Calcolare il **Totale** per ogni riga (`Quantità * Prezzo`).
4. Salvare il report pulito in un nuovo file JSON.

Soluzione Esercizio 1

```
import pandas as pd

# 1. Carico
df = pd.read_csv("vendite_raw.csv")

# 2. Pulizia (Tengo solo prezzi positivi)
df_clean = df[df["PrezzoUnitario"] > 0].copy()

# 3. Calcolo
df_clean["TotaleRiga"] = df_clean["Quantità"] * df_clean["PrezzoUnitario"]

# Calcolo fatturato totale
fatturato_totale = df_clean["TotaleRiga"].sum()
print(f"Fatturato Totale: {fatturato_totale}€")

# 4. Export in JSON
df_clean.to_json("report_vendite.json", orient="records", indent=4)
```

Esercizio 2: Analisi Log di Sistema

Scenario: Hai un file `server.log` con righe nel formato:

```
2024-01-15 10:30:45 INFO User login: mario@email.com  
2024-01-15 10:31:22 ERROR Database connection failed  
2024-01-15 10:32:01 WARNING High memory usage: 85%
```

Obiettivo:

1. Leggere il file di log
2. Contare quanti messaggi per ogni livello (INFO, ERROR, WARNING)
3. Salvare gli ERROR in un file separato

Soluzione Esercizio 2

```
from pathlib import Path
from collections import Counter

log_path = Path("server.log")
conteggio = Counter()
errori = []

with open(log_path, "r", encoding="utf-8") as f:
    for riga in f:
        parti = riga.split()
        if len(parti) ≥ 3:
            livello = parti[2] # INFO, ERROR, WARNING
            conteggio[livello] += 1
            if livello == "ERROR":
                errori.append(riga)

print("Conteggio per livello:")
for livello, count in conteggio.items():
    print(f" {livello}: {count}")

# Salva errori
Path("errori.log").write_text("".join(errori), encoding="utf-8")
```

Esercizio 3: API e Salvataggio

Scenario: Scaricare dati da un'API pubblica e salvarli.

Obiettivo:

1. Scaricare la lista utenti da `https://jsonplaceholder.typicode.com/users`
2. Estrarre nome, email e città
3. Salvare in CSV

Soluzione Esercizio 3

```
import requests
import pandas as pd

# 1. Scarica dati
url = "https://jsonplaceholder.typicode.com/users"
response = requests.get(url)

if response.ok:
    utenti = response.json()

    # 2. Estrai campi
    dati_estratti = []
    for u in utenti:
        dati_estratti.append({
            "nome": u["name"],
            "email": u["email"],
            "città": u["address"]["city"]
        })

    # 3. Crea DataFrame e salva
    df = pd.DataFrame(dati_estratti)
    df.to_csv("utenti.csv", index=False)
    print(f"Salvati {len(df)} utenti")
else:
    print(f"Errore: {response.status_code}")
```

Esercizio 4: Report Vendite per Categoria

Obiettivo: Creare un report aggregato per categoria prodotto.

Soluzione esercizio 4

```
import pandas as pd

# Dati di esempio
df = pd.DataFrame({
    "Prodotto": ["Laptop", "Mouse", "Laptop", "Tastiera", "Mouse"],
    "Categoria": ["Elettronica", "Accessori", "Elettronica",
                  "Accessori", "Accessori"],
    "Vendite": [1200, 25, 1100, 80, 30]
})

# Report per categoria
report = df.groupby("Categoria").agg({
    "Vendite": ["sum", "mean", "count"]
}).round(2)

report.columns = ["Totale", "Media", "N_Vendite"]
print(report)

# Salva report
report.to_excel("report_categorie.xlsx")
```

Riepilogo del Modulo

Argomento	Concetti Chiave
venv	Isolamento progetti, <code>activate</code> , <code>deactivate</code>
pip	<code>install</code> , <code>freeze</code> , <code>requirements.txt</code>
File I/O	<code>open()</code> , <code>with</code> , modalità r/w/a
CSV	<code>csv.reader/writer</code> , <code>DictReader</code>
JSON	<code>json.load/dump</code> , <code>loads/dumps</code>
requests	<code>get()</code> , <code>post()</code> , <code>.json()</code>
Pandas	DataFrame, <code>read_csv</code> , filtri, groupby

⚠ Errori Comuni: File e Dati

Errore	Problema	Soluzione
Encoding errato	<code>UnicodeDecodeError</code>	Specifica <code>encoding="utf-8"</code>
Path separator	<code>\</code> vs <code>/</code> Windows/Linux	Usa <code>pathlib.Path</code>
File non chiuso	Resource leak	Usa sempre <code>with open()</code>
JSON non serializzabile	<code>TypeError</code> per oggetti custom	Converti in dict prima

```
# ❌ Problemi comuni
open("file.txt") # Nessun encoding, non chiuso

# ✅ Corretto
from pathlib import Path
with open(Path("dati") / "file.txt", encoding="utf-8") as f:
    contenuto = f.read()
```



Progetto Cross-Modulo: Rubrica Contatti

Parte 5: Persistenza su File (FINALE)

Soluzione Progetto Cross-Modulo

```
import json
from pathlib import Path

class Rubrica:
    def __init__(self, file_path="rubrica.json"):
        self.file_path = Path(file_path)
        self.contatti = self._carica()

    def _carica(self):
        """Carica contatti da file JSON."""
        if self.file_path.exists():
            return json.loads(self.file_path.read_text(encoding="utf-8"))
        return []

    def salva(self):
        """Salva contatti su file JSON."""
        self.file_path.write_text(
            json.dumps(self.contatti, indent=2, ensure_ascii=False),
            encoding="utf-8"
        )

    def aggiungi(self, nome, telefono, email=""):
        self.contatti.append({"nome": nome, "tel": telefono, "email": email})
        self.salva()

# La rubrica ora persiste tra le esecuzioni!
rubrica = Rubrica()
rubrica.aggiungi("Mario", "333-1234567", "mario@email.com")
```



Congratulazioni!

Hai completato il **Corso Python Base!**

Cosa hai imparato:

- Variabili, tipi di dati, operatori
- Strutture dati: liste, dict, tuple, set
- Controllo di flusso e funzioni
- OOP e gestione eccezioni
- File I/O, API e analisi dati

Cosa vedremo nei prossimi moduli:

- Approfondire Pandas e Data Science
- Web development con Flask/Django